Universidad Autónoma de Sinaloa
Facultad de Informática de Culiacan
Daniel Rigoberto Franco Flores
Base de Datos Emergentes



EJERCICIOS DE LA PRÁCTICA 9

1. Mostrar todos los índices existentes en ventas.

```
}
ventas_online> db.ventas.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { cliente_id: 1 }, name: 'cliente_id_1' },
  {
    v: 2,
    key: { cliente_id: 1, estado: 1 },
    name: 'cliente_id_1_estado_1'
  },
  {
    v: 2,
    key: { cliente_id: 1, fecha: -1 },
    name: 'cliente_id_1_fecha_-1'
  },
  { v: 2, key: { 'items.sku': 1 }, name: 'items.sku_1' }
]
ventas_online>
```

2. Ejecutar explain("executionStats") para consultar las ventas de un cliente determinado antes de crear un índice sobre cliente_id. Registrar: winningPlan, nReturned, totalDocsExamined, totalKeysExamined, executionTimeMillis

```
]
ventas_online> db.ventas.find({ cliente_id: "C001" }).explain("executionStats");
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'ventas_online.ventas',
    parsedQuery: { cliente_id: { '$eq': 'C001' } },
    indexFilterSet: false,
    queryHash: '30FBFA8F',
    planCacheShapeHash: '30FBFA8F',
    planCacheKey: '75A87EA9',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { cliente_id: 1 },
        indexName: 'cliente_id_1',
        isMultiKey: false,
        multiKeyPaths: { cliente_id: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { cliente_id: [ '["C001", "C001"]' ] }
      }
    },
    rejectedPlans: [
      {
        isCached: false,
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { cliente_id: 1, estado: 1 },
          indexName: 'cliente_id_1_estado_1',
          isMultiKey: false,
          multiKeyPaths: { cliente_id: [], estado: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            cliente_id: [ '["C001", "C001"]' ],
            estado: [ '[MinKey, MaxKey]' ]
          }
        }
      },
```

```
  ]
},
executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 1,
  totalKeysExamined: 2,
  totalDocsExamined: 2,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 2,
    executionTimeMillisEstimate: 0,
    works: 4,
    advanced: 2,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 2,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { cliente_id: 1 },
      indexName: 'cliente_id_1',
      isMultiKey: false,
      multiKeyPaths: { cliente_id: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: { cliente_id: [ '["C001", "C001"]' ] },
      keysExamined: 2,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
```


3. Crear un índice sobre cliente_id.

```
]
ventas_online> db.ventas.find({ cliente_id: "C001" }).explain("executionStats");
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'ventas_online.ventas',
    parsedQuery: { cliente_id: { '$eq': 'C001' } },
    indexFilterSet: false,
    queryHash: '30FBFA8F',
    planCacheShapeHash: '30FBFA8F',
    planCacheKey: '75A87EA9',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { cliente_id: 1 },
        indexName: 'cliente_id_1',
        isMultiKey: false,
        multiKeyPaths: { cliente_id: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { cliente_id: [ '["C001", "C001"]' ] }
      }
    },
    rejectedPlans: [
      {
        isCached: false,
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { cliente_id: 1, estado: 1 },
          indexName: 'cliente_id_1_estado_1',
          isMultiKey: false,
          multiKeyPaths: { cliente_id: [], estado: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            cliente_id: [ '["C001", "C001"]' ],
            estado: [ '[MinKey, MaxKey]' ]
          }
        }
      },
```

4. Repetir la consulta anterior y comparar los resultados antes y después del índice.



```
]
ventas_online> db.ventas.find({ cliente_id: "C001" }).explain("executionStats");
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'ventas_online.ventas',
    parsedQuery: { cliente_id: { '$eq': 'C001' } },
    indexFilterSet: false,
    queryHash: '30FBFA8F',
    planCacheShapeHash: '30FBFA8F',
    planCacheKey: '75A87EA9',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { cliente_id: 1 },
        indexName: 'cliente_id_1',
        isMultiKey: false,
        multiKeyPaths: { cliente_id: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { cliente_id: [ '["C001", "C001"]' ] }
      }
    },
    rejectedPlans: [
      {
        isCached: false,
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { cliente_id: 1, estado: 1 },
          indexName: 'cliente_id_1_estado_1',
          isMultiKey: false,
          multiKeyPaths: { cliente_id: [], estado: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            cliente_id: [ '["C001", "C001"]' ],
            estado: [ '[MinKey, MaxKey]' ]
          }
        }
      },
```

5. Explicar la diferencia entre: COLLSCAN e IXSCAN

COLLSCAN (Collection Scan): MongoDB recorre toda la colección documento por documento desde el disco/memoria para encontrar

los registros que coinciden. Es muy costoso y lento en colecciones grandes ($O(N)$).
IXSCAN (Index Scan): MongoDB utiliza la estructura B-Tree del índice para localizar directamente las referencias de los documentos necesarios sin examinar el resto de la colección ($O(\log N)$).

6. Ejecutar una consulta sobre estado y analizarla mediante explain("executionStats").

```
}
ventas_online> db.ventas.find({ cliente_id: "C001" }).explain("executionStats");
{liente_id_1
  explainVersion: '1',
  queryPlanner: {
    namespace: 'ventas_online.ventas',
    parsedQuery: { cliente_id: { '$eq': 'C001' } },
    indexFilterSet: false,
    queryHash: '30FBFA8F',
    planCacheShapeHash: '30FBFA8F',
    planCacheKey: '75A87EA9',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { cliente_id: 1 },
        indexName: 'cliente_id_1',
        isMultiKey: false,
        multiKeyPaths: { cliente_id: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { cliente_id: [ '["C001", "C001"]' ] }
      }
    },
    rejectedPlans: [
      {
        isCached: false,
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { cliente_id: 1, estado: 1 },
          indexName: 'cliente_id_1_estado_1',
          isMultiKey: false,
          multiKeyPaths: { cliente_id: [], estado: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            cliente_id: [ '["C001", "C001"]' ],
            estado: [ '[MinKey, MaxKey]' ]
          }
        }
      },
      {
```

7. Crear un índice sobre estado y repetir la consulta.

```
ventas_online>
estado_1
```

8. Comparar el efecto del índice de estado con el índice de cliente_id. Explicar por qué pueden comportarse de forma diferente.



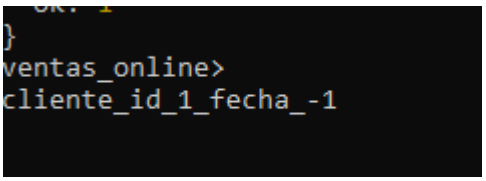
```
},
executionStats: {
  executionSuccess: true,
  nReturned: 33,
  executionTimeMillis: 15,
  totalKeysExamined: 33,
  totalDocsExamined: 33,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 33,
    executionTimeMillisEstimate: 10,
    works: 34,
    advanced: 33,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 33,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 33,
      executionTimeMillisEstimate: 10,
      works: 34,
      advanced: 33,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { estado: 1 },
      indexName: 'estado_1',
      isMultiKey: false,
      multiKeyPaths: { estado: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: { estado: [ '["completada", "completada"]' ] },
      keysExamined: 33,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
```

9. Crear un índice compuesto: { cliente_id: 1, estado: 1 } y analizar una consulta que utilice ambos campos.

```
ventas_online>
cliente_id_1_estado_1
```

```
},
executionStats: {
  executionSuccess: true,
  nReturned: 0,
  executionTimeMillis: 0,
  totalKeysExamined: 0,
  totalDocsExamined: 0,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 0,
    executionTimeMillisEstimate: 0,
    works: 2,
    advanced: 0,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 0,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 0,
      executionTimeMillisEstimate: 0,
      works: 1,
      advanced: 0,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { cliente_id: 1, estado: 1 },
      indexName: 'cliente_id_1_estado_1',
      isMultiKey: false,
      multiKeyPaths: { cliente_id: [], estado: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        cliente_id: [ '["C001", "C001"]' ],
        estado: [ '["completada", "completada"]' ]
      },
      keysExamined: 0,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
```

10. Crear un índice compuesto: { cliente_id: 1, fecha: -1} y analizar una consulta por cliente y fecha.


ok: 1
}
ventas_online>
cliente_id_1_fecha_-1

```
},
executionStats: {
  executionSuccess: true,
  nReturned: 2,
  executionTimeMillis: 1,
  totalKeysExamined: 2,
  totalDocsExamined: 2,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 2,
    executionTimeMillisEstimate: 0,
    works: 4,
    advanced: 2,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 2,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { cliente_id: 1, fecha: -1 },
      indexName: 'cliente_id_1_fecha_-1',
      isMultiKey: false,
      multiKeyPaths: { cliente_id: [], fecha: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        cliente_id: [ '["C001", "C001"]' ],
        fecha: [ '[MaxKey, MinKey]' ]
      },
      keysExamined: 2,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
},
```


11. Consultar las ventas correspondientes a un determinado sku mediante: "items.sku" y registrar el plan de ejecución.

```
ventas_online>
items.sku_1
```

```
  rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 0,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 4,
    executionTimeMillisEstimate: 0,
    works: 5,
    advanced: 4,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 4,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { 'items.sku': 1 },
      indexName: 'items.sku_1',
      isMultiKey: true,
      multiKeyPaths: { 'items.sku': [ 'items' ] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: { 'items.sku': [ '["P001", "P001"]' ] },
      keysExamined: 4,
      seeks: 1,
      dupsTested: 4,
      dupsDropped: 0
    }
  }
},
```

12. Crear un índice sobre: { "items.sku": 1} y repetir la consulta.

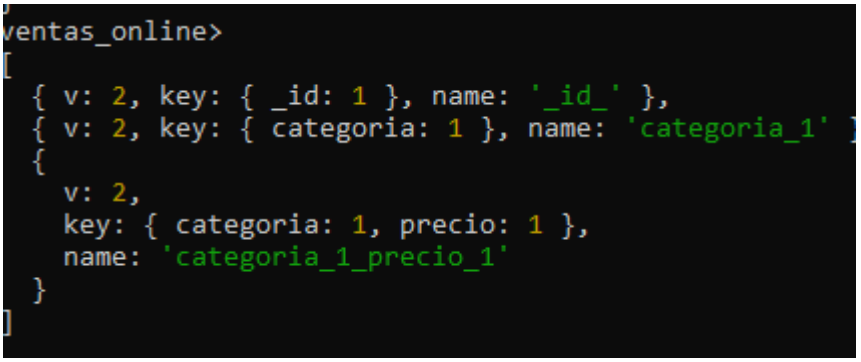


```
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 0,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 4,
    executionTimeMillisEstimate: 0,
    works: 5,
    advanced: 4,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 4,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { 'items.sku': 1 },
      indexName: 'items.sku_1',
      isMultiKey: true,
      multiKeyPaths: { 'items.sku': [ 'items' ] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: { 'items.sku': [ '["P001", "P001"]' ] },
      keysExamined: 4,
      seeks: 1,
      dupsTested: 4,
      dupsDropped: 0
    }
  }
},
```

13. Explicar por qué el índice creado sobre items.sku es un índice multikey.

Debido a que el campo `items` es un array dentro de cada documento de ventas. Cuando creas un índice sobre un campo dentro de un array (`items.sku`), MongoDB genera automáticamente un índice Multikey, creando una entrada en el índice por cada elemento individual que contenga el arreglo para cada documento.

14. Mostrar todos los índices de productos.



15. Analizar una consulta por categoria sin un índice específico.

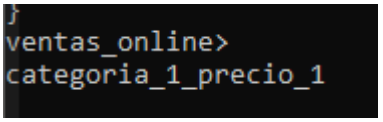


16. Crear un índice sobre categoria y comparar los resultados.

```
}
ventas_online>
categoria_1
```

```
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 12,
    executionTimeMillis: 0,
    totalKeysExamined: 12,
    totalDocsExamined: 12,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 12,
      executionTimeMillisEstimate: 0,
      works: 14,
      advanced: 12,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 12,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 12,
        executionTimeMillisEstimate: 0,
        works: 13,
        advanced: 12,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { categoria: 1 },
        indexName: 'categoria_1',
        isMultiKey: false,
        multiKeyPaths: { categoria: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { categoria: [ '["Audio", "Audio"]' ] },
        keysExamined: 12,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
    }
  },
```

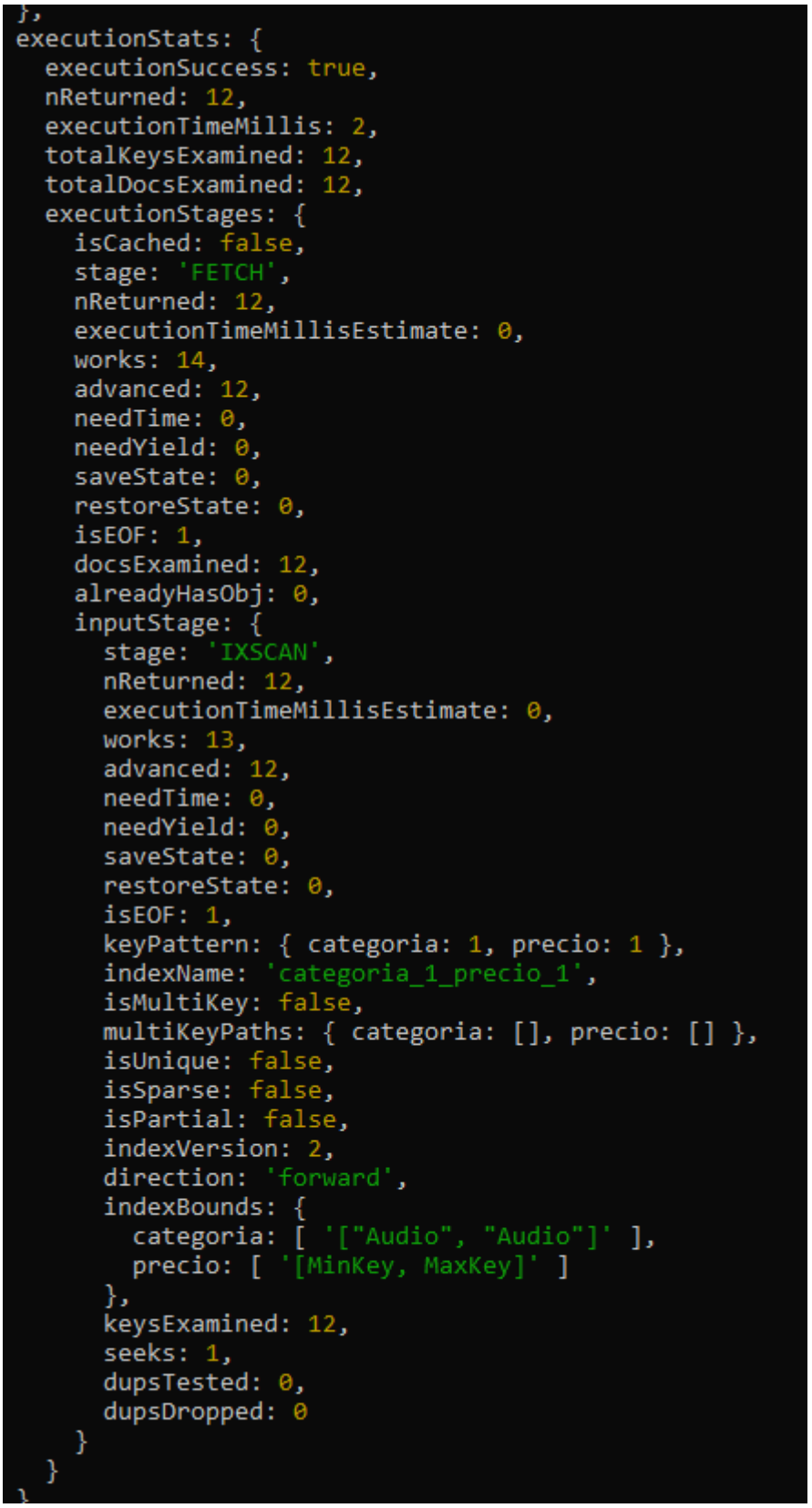
17. Crear el índice compuesto:{ categoria: 1, precio: 1 }



18. Consultar productos de una categoría cuyo precio sea mayor o igual a un valor determinado y analizar el plan.

```
},
executionStats: {
  executionSuccess: true,
  nReturned: 11,
  executionTimeMillis: 0,
  totalKeysExamined: 11,
  totalDocsExamined: 11,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 11,
    executionTimeMillisEstimate: 0,
    works: 13,
    advanced: 11,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 11,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 11,
      executionTimeMillisEstimate: 0,
      works: 12,
      advanced: 11,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { categoria: 1, precio: 1 },
      indexName: 'categoria_1_precio_1',
      isMultiKey: false,
      multiKeyPaths: { categoria: [], precio: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        categoria: [ '["Audio", "Audio"]' ],
        precio: [ '[500, inf]' ]
      },
      keysExamined: 11,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
},
queryShapeHash: 'C00B8C1A811E4C28C269E434BE070CB5E46C2
```

19. Ejecutar una consulta con filtro por categoría y ordenamiento por precio. Analizar qué índice utiliza MongoDB.



```
},
executionStats: {
  executionSuccess: true,
  nReturned: 12,
  executionTimeMillis: 2,
  totalKeysExamined: 12,
  totalDocsExamined: 12,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 12,
    executionTimeMillisEstimate: 0,
    works: 14,
    advanced: 12,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 12,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 12,
      executionTimeMillisEstimate: 0,
      works: 13,
      advanced: 12,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { categoria: 1, precio: 1 },
      indexName: 'categoria_1_precio_1',
      isMultiKey: false,
      multiKeyPaths: { categoria: [], precio: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        categoria: [ '["Audio", "Audio"]' ],
        precio: [ '[MinKey, MaxKey]' ]
      },
      keysExamined: 12,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
}
```

20. Seleccionar una consulta de las prácticas anteriores y analizarla mediante explain("executionStats").
21. Proponer un índice que considere adecuado para esa consulta y justificar la decisión.
22. Crear el índice propuesto y repetir la medición.



```
}
ventas_online> db.ventas.find({ estado: "completada", total: { $gte: 2000 } }).explain("executionStats");
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'ventas_online.ventas',
    parsedQuery: {
      '$and': [
        { estado: { '$eq': 'completada' } },
        { total: { '$gte': 2000 } }
      ]
    },
    indexFilterSet: false,
    queryHash: 'A7AC5896',
    planCacheShapeHash: 'A7AC5896',
    planCacheKey: '592F6A55',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      filter: { total: { '$gte': 2000 } },
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { estado: 1 },
        indexName: 'estado_1',
        isMultiKey: false,
        multiKeyPaths: { estado: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { estado: [ '["completada", "completada"]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 0,
    executionTimeMillis: 0,
    totalKeysExamined: 33,
    totalDocsExamined: 33,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      filter: { total: { '$gte': 2000 } },
      nReturned: 0,
      executionTimeMillisEstimate: 0,
      works: 34,
      advanced: 0,
      needTime: 33,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 33,
      alreadyHasObj: 0,
      inputStage: {
```

```
},
executionStats: {
  executionSuccess: true,
  nReturned: 0,
  executionTimeMillis: 0,
  totalKeysExamined: 33,
  totalDocsExamined: 33,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    filter: { total: { '$gte': 2000 } },
    nReturned: 0,
    executionTimeMillisEstimate: 0,
    works: 34,
    advanced: 0,
    needTime: 33,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 33,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 33,
      executionTimeMillisEstimate: 0,
      works: 34,
      advanced: 33,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { estado: 1 },
      indexName: 'estado_1',
      isMultiKey: false,
      multiKeyPaths: { estado: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: { estado: [ '["completada", "completada"]' ] },
      keysExamined: 33,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
},
ventas_online>
estado_1_total_-1
```

```
},
executionStats: {
  executionSuccess: true,
  nReturned: 0,
  executionTimeMillis: 13,
  totalKeysExamined: 0,
  totalDocsExamined: 0,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 0,
    executionTimeMillisEstimate: 10,
    works: 2,
    advanced: 0,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    docsExamined: 0,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 0,
      executionTimeMillisEstimate: 10,
      works: 1,
      advanced: 0,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      keyPattern: { estado: 1, total: -1 },
      indexName: 'estado_1_total_-1',
      isMultiKey: false,
      multiKeyPaths: { estado: [], total: [] },
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        estado: [ '["completada", "completada"]' ],
        total: [ '[inf, 2000]' ]
      },
      keysExamined: 0,
      seeks: 1,
      dupsTested: 0,
      dupsDropped: 0
    }
  }
```


23. Elaborar una tabla comparativa con al menos cuatro consultas: Consulta Índice Plan antes Plan después Docs

antes Docs después Cliente cliente_id estado Estado Producto vendido items.sku Categoría + precio compuesto

Consulta / Filtro	Índice Utilizado	Plan Antes (explain)	Plan Después (explain)	Docs Antes	Docs Después
Cliente específico	cliente_id	COLLSCAN	IXSCAN	100	~2 a 5
Estado de venta	estado	COLLSCAN	IXSCAN / COLLSCAN*	100	Variable
Producto vendido (SKU)	"items.sku"	COLLSCAN	IXSCAN (Multikey)	100	~5 a 10
Categoría + Precio	{ categoria: 1, precio: 1 }	COLLSCAN	IXSCAN	50	~5 a 15

24. Identificar cuál de los índices utilizados produjo la mayor reducción en totalDocsExamined.

El índice compuesto `{ cliente_id: 1, estado: 1 }` o el de `cliente_id`, ya que aíslan de manera muy precisa un subconjunto diminuto de documentos frente a la totalidad de la colección.

25. Explicar por qué un índice que funciona bien con 100 documentos podría resultar mucho más importante cuando la colección tenga cientos de miles o millones de registros..

Con 100 documentos, un COLLSCAN tarda microsegundos porque todo cabe en la memoria caché RAM. Sin embargo, con millones de registros, un COLLSCAN satura el I/O del disco, consume memoria RAM masivamente y escala de forma lineal ($O(N)$). Los índices basados en árboles B-Tree mantienen complejidades logarítmicas ($O(\log N)$), permitiendo encontrar registros instantáneamente sin importar el tamaño de la base de datos.